



Atividade Final

RM364457 – Bruno Gomes Nogueira

RM 364668 – João Paulo Martins Rodrigues

Você é responsável pela manutenção e otimização de um banco de dados Oracle para uma aplicação que lida com um grande volume de dados. A tabela `ORDERS` armazena informações sobre pedidos de uma empresa de e-commerce e contém milhões de registros.

A tabela `ORDERS` tem a seguinte estrutura:

```
CREATE TABLE ORDERS (  
    ORDER_ID      NUMBER CONSTRAINT ORDERS_PK PRIMARY KEY,  
    CUSTOMER_ID   NUMBER,  
    ORDER_DATE    DATE,  
    SHIP_DATE     DATE,  
    STATUS        VARCHAR2(20),  
    TOTAL_AMOUNT  NUMBER  
);
```

Vamos popular a tabela `ORDERS` com uma grande quantidade de dados. A seguir está o script para a carga de dados:

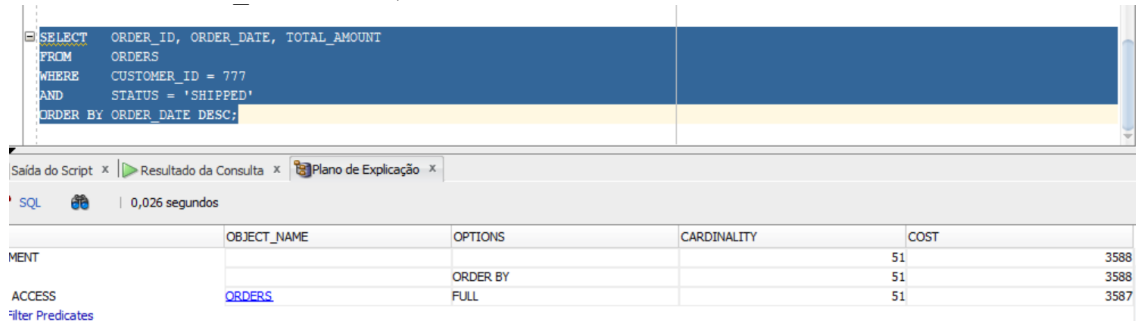
```
BEGIN  
    FOR i IN 1..2000000 LOOP  
        INSERT INTO ORDERS  
        VALUES (  
            i,  
            MOD(i, 10000) + 1,  
            TRUNC(SYSDATE - DBMS_RANDOM.VALUE(0, 730)),  
            TRUNC(SYSDATE - DBMS_RANDOM.VALUE(0, 730)),  
            CASE  
                WHEN MOD(i, 4) = 0 THEN 'SHIPPED'  
                WHEN MOD(i, 4) = 1 THEN 'PENDING'  
                WHEN MOD(i, 4) = 2 THEN 'CANCELLED'  
                ELSE 'RETURNED'  
            END,  
            ROUND(DBMS_RANDOM.VALUE(10, 5000), 2)  
        );  
  
        IF MOD(i, 10000) = 0 THEN  
            COMMIT;  
        END IF;  
    END LOOP;  
    COMMIT;  
END;  
/
```

Consultas Problemáticas

Duas consultas específicas estão apresentando problemas de performance:

1. Consulta que busca pedidos por cliente e por status:

```
SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT
FROM   ORDERS
WHERE  CUSTOMER_ID = 777
AND    STATUS = 'SHIPPED'
ORDER BY ORDER_DATE DESC;
```

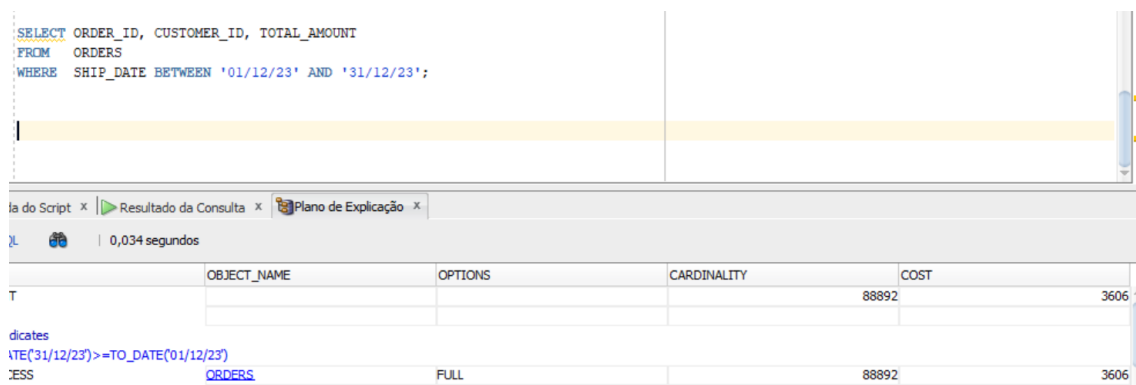


	OBJECT_NAME	OPTIONS	CARDINALITY	COST
MENT			51	3588
		ORDER BY	51	3588
ACCESS	ORDERS	FULL	51	3587

filter Predicates

2. Consulta que busca pedidos por data de envio: (DESAFIO)

```
SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT
FROM   ORDERS
WHERE  SHIP_DATE BETWEEN '01-DEC-23' AND '31-DEC-23';
```



	OBJECT_NAME	OPTIONS	CARDINALITY	COST
T			88892	3606
dicates				
ATE('31/12/23')>=TO_DATE('01/12/23')				
CESS	ORDERS	FULL	88892	3606

Atividade

Você deve criar índices apropriados ou, até mesmo reescrever a instrução para melhorar a performance dessas consultas. Para isso, siga os seguintes passos:

a. Análise Inicial (2 pontos)

Explique a razão pela qual essas consultas podem estar lentas. Considere fatores como tamanho da tabela, colunas envolvidas nas cláusulas WHERE e ORDER BY.

R.: As duas consultas estão lentas por alguns motivos:

- **Volume de dados:** A tabela ORDERS tem 2 milhões de registros, o que torna qualquer varredura completa (full scan) muito custosa/demorada.
- **Filtros nas cláusulas WHERE:**
 - A primeira consulta filtra por CUSTOMER_ID e STATUS, depois ordena por ORDER_DATE DESC.
 - A segunda consulta filtra por intervalo de datas em SHIP_DATE.
- **Falta de índices:** Sem índices nas colunas usadas nos filtros (CUSTOMER_ID, STATUS, SHIP_DATE), o banco precisa escanear a tabela inteira.
- **Ordenação:** A ordenação por ORDER_DATE também pode ser lenta se não houver suporte por índice.

b. Criação de Índices (2 pontos)

Proponha e crie índices que possam melhorar a performance das consultas mencionadas. Escreva os comandos SQL necessários para criar esses índices.

-- Índice para a primeira consulta

```
CREATE INDEX IDX_ORDERS_CUSTOMER_STATUS_DATE
```

```
ON ORDERS (CUSTOMER_ID, STATUS, ORDER_DATE DESC);
```

```
-- Índice para a primeira consulta
CREATE INDEX IDX_ORDERS_CUSTOMER_STATUS_DATE
ON ORDERS (CUSTOMER_ID, STATUS, ORDER_DATE DESC);

SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT
FROM ORDERS
WHERE CUSTOMER_ID = 777
AND STATUS = 'SHIPPED'
ORDER BY ORDER_DATE DESC;
```

OBJECT_NAME	OPTIONS	CARDINALITY	COST
ORDERS	BY INDEX ROWID	51	54
IDX_ORDERS_CUSTOMER_STATUS_D...	RANGE SCAN	51	3

-- Índice para a segunda consulta

CREATE INDEX IDX_ORDERS_SHIP_DATE_OPT

ON ORDERS (SHIP_DATE, ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT);

```
-- Índice para a segunda consulta
CREATE INDEX IDX_ORDERS_SHIP_DATE_OPT
ON ORDERS (SHIP_DATE, ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT);

SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT
FROM ORDERS
WHERE SHIP_DATE BETWEEN '01/12/23' AND '31/12/23';
```

la do Script x Resultado da Consulta x Plano de Explicação x

0,079 segundos

	OBJECT_NAME	OPTIONS	CARDINALITY	COST
T				88892
				413

icates

ATE('31/12/23')>=TO_DATE('01/12/23')

	IDX_ORDERS_SHIP_DATE_OPT	RANGE SCAN		88892
				413

ss Predicates

OPTIONS=RANGE SCAN

Validação (1 ponto)

Mostre como validar a eficácia dos índices criados. Utilize a função (F10) EXPLAIN PLAN para comparar os planos de execução antes e depois da criação dos índices.

Primeira consulta

Antes

```

SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT
FROM   ORDERS
WHERE  CUSTOMER_ID = 777
AND    STATUS = 'SHIPPED'
ORDER BY ORDER_DATE DESC;

```

Saída do Script x | Resultado da Consulta x | Plano de Explicação x

SQL  | 0,026 segundos

	OBJECT_NAME	OPTIONS	CARDINALITY	COST	
MENT				51	3588
		ORDER BY		51	3588
ACCESS	ORDERS	FULL		51	3587

[Filter Predicates](#)

 **PLAN_TABLE_OUTPUT**

```

1 Plan hash value: 36248429
2
3 -----
4 | Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
5 -----
6 | 0 | SELECT STATEMENT | | | 51 | 1632 | 3588 (1) | 00:00:01 |
7 | 1 | SORT ORDER BY | | | 51 | 1632 | 3588 (1) | 00:00:01 |
8 |* 2 | TABLE ACCESS FULL| ORDERS | 51 | 1632 | 3587 (1) | 00:00:01 |
9 -----
10
11 Predicate Information (identified by operation id):
12 -----
13
14 2 - filter("CUSTOMER_ID">777 AND "STATUS">'SHIPPED')

```

Depois

```
-- índice para a primeira consulta
CREATE INDEX IDX_ORDERS_CUSTOMER_STATUS_DATE
ON ORDERS (CUSTOMER_ID, STATUS, ORDER_DATE DESC);

SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT
FROM ORDERS
WHERE CUSTOMER_ID = 777
AND STATUS = 'SHIPPED'
ORDER BY ORDER_DATE DESC;
```

OBJECT_NAME	OPTIONS	CARDINALITY	COST
ORDERS	BY INDEX ROWID	51	54
ORDERS	BY INDEX ROWID	51	54
ORDERS	RANGE SCAN	51	3

```
Plan hash value: 2507631767
```

ID	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		51	1632	54 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	ORDERS	51	1632	54 (0)	00:00:01
* 2	INDEX RANGE SCAN	IDX_ORDERS_CUSTOMER_STATUS_DATE	51		3 (0)	00:00:01

Predicate Information (identified by operation id):

```
2 - access("CUSTOMER_ID"=777 AND "STATUS"='SHIPPED')
```

Segunda Consulta

Antes

```
SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT
FROM ORDERS
WHERE SHIP_DATE BETWEEN '01/12/23' AND '31/12/23';
```

0,034 segundos

	OBJECT_NAME	OPTIONS	CARDINALITY	COST	
T				88892	3606
dicates					
TE('31/12/23')>=TO_DATE('01/12/23')					
ESS	ORDERS	FULL		88892	3606

PLAN_TABLE_OUTPUT

```
1 Plan hash value: 4130335093
2
3 -----
4 | Id | Operation          | Name | Rows | Bytes | Cost (%CPU)| Time     |
5 -----
6 |  0 | SELECT STATEMENT   |      | 88892 | 1996K | 3606  (2)| 00:00:01 |
7 |*  1 | FILTER              |      |      |      |          |          |
8 |*  2 | TABLE ACCESS FULL| ORDERS | 88892 | 1996K | 3606  (2)| 00:00:01 |
9 -----
10
11 Predicate Information (identified by operation id):
12 -----
13
14 1 - filter(TO_DATE('31/12/23')>=TO_DATE('01/12/23'))
15 2 - filter("SHIP_DATE"<='31/12/23' AND "SHIP_DATE">='01/12/23')
```


Depois

```
-- índice para a segunda consulta
CREATE INDEX IDX_ORDERS_SHIP_DATE_OPT
ON ORDERS (SHIP_DATE, ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT);

SELECT ORDER_ID, CUSTOMER_ID, TOTAL_AMOUNT
FROM ORDERS
WHERE SHIP_DATE BETWEEN '01/12/23' AND '31/12/23';
```

Resultado da Consulta x Plano de Explicação x

0,079 segundos

	OBJECT_NAME	OPTIONS	CARDINALITY	COST	
T				88892	413
icates					
TE('31/12/23')>=TO_DATE('01/12/23')	IDX_ORDERS_SHIP_DATE_OPT	RANGE SCAN		88892	413
ss Predicates				OPTIONS=RANGE SCAN	

PLAN_TABLE_OUTPUT

```
1 Plan hash value: 4149859647
2
3 -----
4 | Id | Operation          | Name                | Rows | Bytes | Cost (%CPU)| Time     |
5 -----
6 |  0 | SELECT STATEMENT    |                     | 88892 | 1996K |  413  (1)| 00:00:01 |
7 |*  1 |  FILTER             |                     |      |      |           |          |
8 |*  2 |    INDEX RANGE SCAN | IDX_ORDERS_SHIP_DATE_OPT | 88892 | 1996K |  413  (1)| 00:00:01 |
9 -----
10
11 Predicate Information (identified by operation id):
12 -----
13
14    1 - filter(TO_DATE('31/12/23')>=TO_DATE('01/12/23'))
15    2 - access("SHIP_DATE">='01/12/23' AND "SHIP_DATE"<='31/12/23')
```

3. Após a aquisição de outra empresa, todos os pedidos gerados pela empresa adquirida precisam ser carregados dentro da tabela ORDERS. Eles são transferidos em formato texto tipo CSV para a matriz, onde tem o banco de dados com a tabela ORDERS.

Monte uma solução que sem usar outra ferramenta carregue estes dados dentro da tabela ORDERS.

Observação: Os registros que tiverem o mesmo ID (order_id) devem ser inseridos em outra tabela ORDERS_REVIEW com a mesma estrutura de ORDERS.

(5 pontos)

-- Criar diretório

```
CREATE OR REPLACE DIRECTORY ORDERS_EXT_DIR AS '/opt/oracle/ext';  
GRANT READ, WRITE ON DIRECTORY ORDERS_EXT_DIR TO RM364457;
```

-- Script para leitura e inserção:

```
CREATE TABLE EXT_ORDERS (  
  ORDER_ID_S   VARCHAR2(20),  
  CUSTOMER_ID_S VARCHAR2(20),  
  ORDER_DATE_S VARCHAR2(10),  
  SHIP_DATE_S  VARCHAR2(10),  
  STATUS_S     VARCHAR2(20),  
  TOTAL_AMOUNT_S VARCHAR2(32)  
)  
  
ORGANIZATION EXTERNAL (  
  TYPE ORACLE_LOADER  
  DEFAULT DIRECTORY ORDERS_EXT_DIR  
  ACCESS PARAMETERS (  
    RECORDS DELIMITED BY NEWLINE  
    SKIP 1  
    CHARACTERSET UTF8  
    BADFILE ORDERS_EXT_DIR:'ext_orders.bad'
```

```

LOGFILE ORDERS_EXT_DIR:'ext_orders.log'

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"' LRTRIM

MISSING FIELD VALUES ARE NULL

( ORDER_ID_S, CUSTOMER_ID_S, ORDER_DATE_S, SHIP_DATE_S, STATUS_S,
TOTAL_AMOUNT_S )

)

LOCATION ('orders_acquired_sample.csv')

)

REJECT LIMIT UNLIMITED;


SELECT * FROM EXT_ORDERS;


BEGIN

INSERT FIRST

    WHEN dup_flag = 1 THEN

        INTO ORDERS_REVIEW (ORDER_ID, CUSTOMER_ID, ORDER_DATE, SHIP_DATE,
STATUS, TOTAL_AMOUNT)

        VALUES (order_id, customer_id, order_date, ship_date, status, total_amount)

    WHEN dup_flag = 0 THEN

        INTO ORDERS (ORDER_ID, CUSTOMER_ID, ORDER_DATE, SHIP_DATE, STATUS,
TOTAL_AMOUNT)

        VALUES (order_id, customer_id, order_date, ship_date, status, total_amount)

SELECT

    TO_NUMBER(e.ORDER_ID_S) AS order_id,

    TO_NUMBER(e.CUSTOMER_ID_S) AS customer_id,

    TO_DATE(e.ORDER_DATE_S,'YYYY-MM-DD') AS order_date,

    TO_DATE(e.SHIP_DATE_S,'YYYY-MM-DD') AS ship_date,

    e.STATUS_S AS status,


    TO_NUMBER(e.TOTAL_AMOUNT_S,'9999999990D99','NLS_NUMERIC_CHARACTERS=.,')
AS total_amount,

```

```
    CASE WHEN t.ORDER_ID IS NULL THEN 0 ELSE 1 END AS dup_flag  
FROM EXT_ORDERS e  
LEFT JOIN ORDERS t ON t.ORDER_ID = TO_NUMBER(e.ORDER_ID_S);  
END;
```